

1. Descrizione del funzionamento del programma prima dell'esecuzione

Dall'analisi del codice del programma, che si presenta in linguaggio di programmazione ad alto livello C, sono invocate le seguenti operazioni in sequenza:

1.1. Librerie definite per l'inclusione nel codice

In riga 1, attraverso la direttiva `#include`, è inserita il codice della libreria `stdio.h` di gestione dell'input e dell'output in un ambiente di linea di comando.

```
1 | #include <stdio.h>
```

1.2. Dichiarazioni delle funzioni

Nel codice è definita la funzione principale `main` dalla riga 3 alla riga 51. Questa funzione è richiamata al momento dell'esecuzione del programma, e ritorna un intero di valore 0, il quale è tradizionalmente usato dal sistema operativo come indice di esecuzione terminata senza eccezioni/errori.

1.3. Dichiarazioni dei tipi di dati

All'interno del blocco di codice della funzione principale `main`, ovvero nelle righe 6 e 7 sono allocati sullo stack un array statico `vector` di 10 posizioni disponibile che possono contenere esclusivamente dati di tipo intero (`int`), e una variabile di appoggio di tipo intero (`int`) denominata `swap_var`.

```
6 | int vector[10], i, j, k;  
7 | int swap_var;
```

1.4. Richiesta di input all'utente

Nell'esecuzione delle linee del codice successive compilate, si prevede che l'utente inserisca esattamente 10 interi, uno per ogni posizione dell'array (dalla posizione 0 alla 9). La variabile `c` è solo un contatore estetico per mostrare all'utente indici da 1 a 10.

Sempre all'interno della funzione principale `main`, dalla riga 11 alla riga 16 è definito un ciclo `for` in ripetizione fintanto che la variabile `i` sia inferiore ad una variabile costante di 10, pari alla lunghezza (o dimensione) dell'array statico `vector`. In riga 12 è definita la variabile `c`, di tipo intero, che si presenta come un contatore estetico per mostrare all'utente indici da 1 a 10. A questa variabile (`c`) è assegnato, per ogni stato dell'iterazione, il valore di `i` incrementato di 1, per essere mostrato all'utente tramite la funzione `printf`, in una stringa formattata, in riga 13, per indicare il numero della posizione prevista per l'immissione del numero intero in input, a partire da 1. Questo numero intero di input è letto dalla funzione `scanf` in riga 14, e scritto nell'indirizzo di memoria della posizione `i` dell'array statico `vector`, tramite il simbolo `&`.

```
11 | for (i = 0; i < 10; i++) {  
12 |     int c = i + 1;  
13 |     printf("[%d]:", c);  
14 |     scanf("%d", &vector[i]);  
15 | }
```

E' da notare che il richiamo `scanf("%d", ...)` non esegue alcun controllo sulla quantità dei dati inseriti rispetto alla capacità dell'array. Di conseguenza può definirsi un punto di inizio di una manifestazione di un *buffer overflow*, o BOF.

1.5. Restituzione a linea di comando

Stampa i 10 valori così come sono stati inseriti.

```
19 printf ("Il vettore inserito e':\n");
20 for ( i = 0 ; i < 10 ; i++)
21     {
22         int t= i+1;
23         printf("[%d]: %d", t, vector[i]);
24         printf("\n");
25     }
```

1.6. Fase 4 – Ordinamento (Bubble Sort)

Nell'esecuzione delle linee del codice successive compilate, si prevede l'ordinamento degli elementi del vettore tramite algoritmo di ordinamento *Bubble Sort*.

Sempre all'interno della funzione principale `main`, dalla riga 28 alla riga 39 è definito l'algoritmo di ordinamento *Bubble Sort* (ordinamento a bolle) dove sono organizzati gli elementi di un vettore confrontando coppie di elementi adiacenti e scambiandoli di posizione se non sono nell'ordine corretto.

```
28 for (j = 0 ; j < 10 - 1; j++)
29     {
30         for (k = 0 ; k < 10 - j - 1; k++)
31             {
32                 if (vector[k] > vector[k+1])
33                 {
34                     swap_var=vector[k];
35                     vector[k]=vector[k+1];
36                     vector[k+1]=swap_var;
37                 }
38             }
39     }
```

In questo blocco sono definiti due cicli iterativi nidificati `for` dove:

- Il ciclo esterno `for`, con variabile `j`, gestisce le passate complessive sul vettore. Per un vettore di 10 elementi, sono necessarie al massimo 9 iterazioni principali. Ad ogni passaggio completo, l'elemento maggiore tra quelli non ancora ordinati si sposta ("galleggia") verso la fine della struttura.
- Il ciclo interno `for`, con variabile `k`, esegue un'operazione di confronto tra elementi vicini all'interno della porzione non ancora ordinata del vettore. Il limite superiore `10 - j - 1` si riduce a ogni iterazione esterna perché gli ultimi elementi sono già posizionati correttamente. In questo blocco iterativo, in riga 32, è verificato se l'elemento corrente `vector[k]` è maggiore del successivo `vector[k+1]`, e in caso affermativo, si attiva la logica di scambio, utilizzando la variabile temporanea `swap_var` per riorganizzare i valori senza perderne alcuno durante la sovrascrittura.

Indice di riga	Invocazione	Azione
34	<code>swap_var = vector[k];</code>	salva temporaneamente il valore più grande
35	<code>vector[k] = vector[k+1];</code>	sposta il valore minore nella posizione precedente
36	<code>vector[k+1] = swap_var;</code>	assegna il valore più grande alla posizione successiva

1.7. Fase 5 – Stampa del vettore ordinato

Nell'esecuzione delle linee del codice successive compilate, si prevede la stampa a schermo, ovvero l'inserimento sul buffer stdout della linea di comando, del vettore ordinato dall'algoritmo di *Bubble Sort* esposto nel paragrafo precedente.

Sempre all'interno della funzione principale `main`, nella riga 40, con la funzione `printf` è stampata a schermo una frase iniziale per indicare all'utente che sta per essere visualizzato il contenuto del vettore ormai ordinato.

Mentre è definito un ciclo iterativo `for` nell'intervallo di righe 41-46, che scorre l'intero array da indice 0 a 9 (per un totale di 10 elementi). Per ogni ciclo iterativo di questo, è calcolato l'indice di posizione `g` (con

`int g = j+1`) per l'incremento dello stesso di 1 al fine di mostrare all'utente una numerazione degli elementi partendo da 1 anziché da 0. Infine, nelle righe 44 e 45, ovvero all'interno di ogni iterazione, sono stampati prima il numero della posizione formattato e subito dopo il valore numerico effettivo memorizzato nel vettore alla posizione `vector[j]`, andando ogni volta a capo.

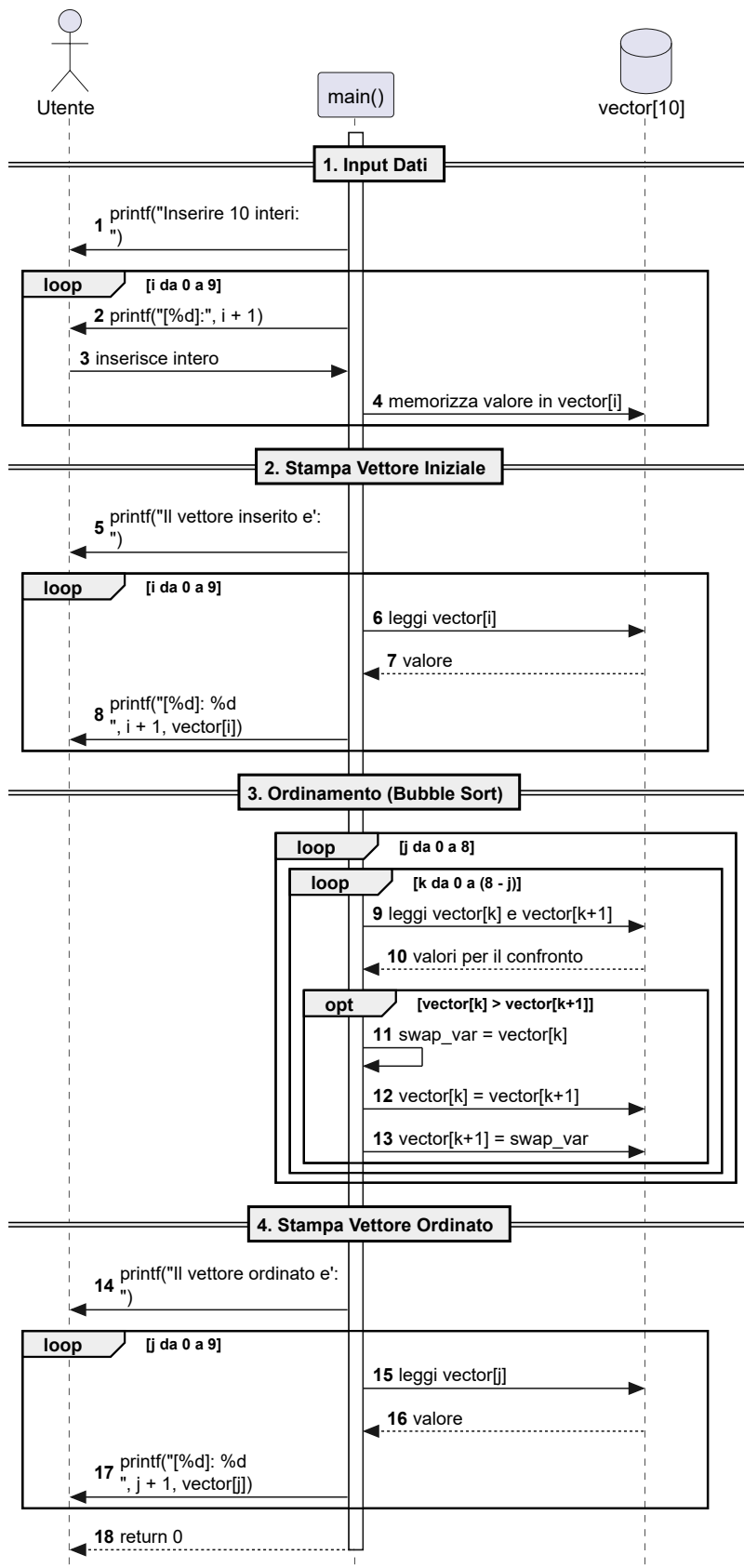
```
40 printf("Il vettore ordinato e':\n");
41 for (j = 0; j < 10; j++)
42     {
43         int g = j+1;
44         printf("[%d]:", g);
45         printf("%d\n", vector[j]);
46     }
```

1.8. Conclusione del programma

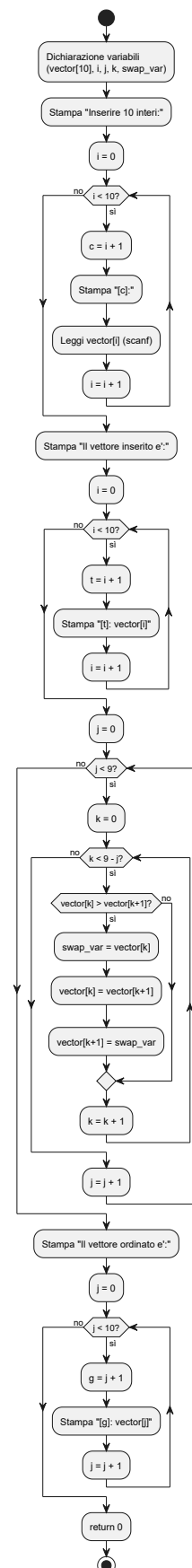
Sempre all'interno della funzione principale `main`, nella riga 48, è ritornato un intero di valore 0. Questo è tradizionalmente usato dal sistema operativo come indice di esecuzione terminata senza eccezioni/errori.

1.9. Diagrammi riassuntivi del flusso

1.9.1. UML sequence diagram



1.9.2. Diagramma di flusso



2. Riproduzione ed esecuzione

2.1. Compilazione

Per la compilazione ed esecuzione, il codice è salvato in un file denominato `exhibit_originale.c` e compilato tramite il software di compilazione *GNU Compiler Collection*, o `gcc`, con il comando:

```
gcc exhibit_originale.c -o exhibit_originale.o
```

2.2. Esecuzione e verifica delle ipotesi

```
(kali㉿kali)~[~]
$ Desktop/exhibit_originale.o
Inserire 10 interi:
[1]:2
[2]:3
[3]:7
[4]:10
[5]:12
[6]:123
[7]:112
[8]:12
[9]:1
[10]:11
Il vettore inserito e':
[1]: 2
[2]: 3
[3]: 7
[4]: 10
[5]: 12
[6]: 123
[7]: 112
[8]: 12
[9]: 1
[10]: 11
Il vettore ordinato e':
[1]:1
[2]:2
[3]:3
[4]:7
[5]:10
[6]:11
[7]:12
[8]:12
[9]:112
[10]:123
```

Dall'esecuzione del programma, che è stato eseguito tramite il comando `./exhibit_originale.o`, si sono notati i seguenti riscontri:

Ipotesi	Riscontro
Il programma accetta esattamente 10 interi	✅ Confermato
L'array ha capienza 10 elementi (indici 0-9)	✅ Confermato
Il Bubble Sort ordina in senso crescente	✅ Confermato
Non ci sono controlli sull'input	✅ Confermato — inserendo caratteri non numerici <code>scanf</code> va in errore silenzioso
Non c'è modo di scrivere oltre i 10 elementi nel codice originale	✅ Confermato — il ciclo <code>for</code> è rigido su <code>i < 10</code>

3. Modifica per Segmentation Fault (BOF)

Con la presente fase, l'obiettivo è la modifica del programma per l'inserimento da parte dell'utente di più di 10 valori, sovrascrivendo la memoria oltre i limiti dell'array `vector[10]`. Questo causerà un errore di *Segmentation Fault* nel momento in cui si scrive su indirizzi di memoria non validi o si corrompe il record di attivazione (*stack frame*).

3.1. Strategia applicata

Per il raggiungimento dell'obiettivo si è sostituito il ciclo `for` rigido con un ciclo `while(1)` (infinito) che continua a leggere input senza controllare i limiti dell'array. Di conseguenza, quando l'utente avrà inserito abbastanza valori da corrompere lo *stack*, il programma si interromperà per errore di segmentazione. Il codice modificato è stato spostato su un file denominato `bof.c`.

Per l'esecuzione e dimostrazione, il programma, denominato `bof.o` è stato compilato tramite il comando:

```
gcc bof.c -o bof.o -fno-stack-protector -z execstack
```

dove sono stati aggiunti gli argomenti `-fno-stack-protector` e `-z execstack`, che, in breve, disabilitano le protezioni dello stack per rendere visibili gli effetti del buffer overflow.

3.2. Esecuzione della strategia

Nell'esecuzione del programma compilato, quando si è superato l'indice 9, si è scritto in una posizione di memoria non appartenente all'array. Nel dettaglio dello stack, l'array `vector[10]` è circondato da:

Indirizzo relativo	Caratteristica contenuta
<code>vector[0..9]</code> (40 byte)	L'array vero e proprio
<code>vector[10]</code>	Altre variabili locali (<code>i</code> , <code>j</code> , <code>k</code> , <code>swap_var</code> , <code>valore</code>)
Dopo	Indirizzo di ritorno (<code>saved EBP/RBP</code> e <code>return address</code>)

Nella sovrascrittura dell'indirizzo di ritorno con un valore non valido, ne è conseguito un errore di segmentazione alla linea di codice `return 0;`.

```
Session Actions Edit View Help
kali@kali: ~$ ./bof.o
Versione BOF - Inserisci valori (CTRL+C per uscire, oppure aspetta il crash):
Array dichiarato: vector[10] sullo stack.
Il ciclo NON ha limiti: scriverà oltre la memoria allocata fino al crash.

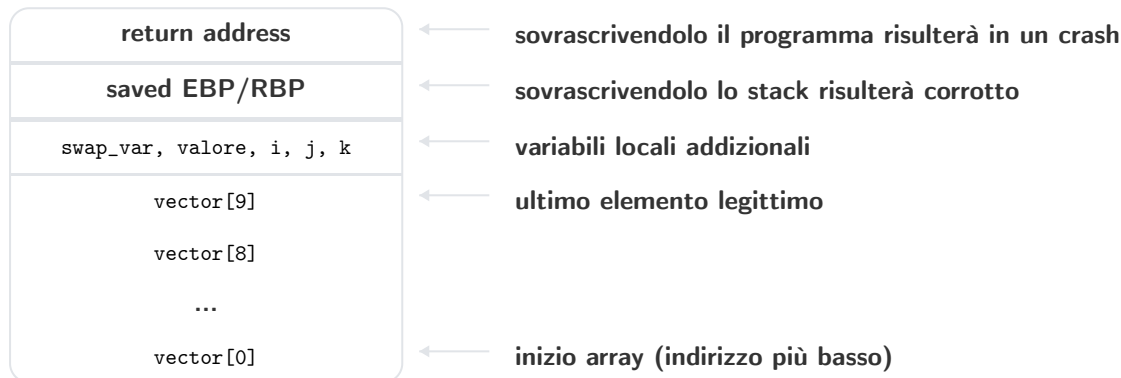
[1]: 10
> Scritto 10 in vector[0] all'indirizzo 0x7ffcd660340
[2]: 20
> Scritto 20 in vector[1] all'indirizzo 0x7ffcd660344
[3]: 30
> Scritto 30 in vector[2] all'indirizzo 0x7ffcd660348
[4]: 40
> Scritto 40 in vector[3] all'indirizzo 0x7ffcd66034c
[5]: 50
> Scritto 50 in vector[4] all'indirizzo 0x7ffcd660350
[6]: 60
> Scritto 60 in vector[5] all'indirizzo 0x7ffcd660354
[7]: 80
> Scritto 80 in vector[6] all'indirizzo 0x7ffcd660358
[8]: 90
> Scritto 90 in vector[7] all'indirizzo 0x7ffcd66035c
[9]: 100
> Scritto 100 in vector[8] all'indirizzo 0x7ffcd660360
[10]: 110
> Scritto 110 in vector[9] all'indirizzo 0x7ffcd660364
[11]: 120
> Scritto 120 in vector[10] all'indirizzo 0x7ffcd660368
[12]: 130
> Scritto 130 in vector[11] all'indirizzo 0x7ffcd66036c
[13]: 140
> Scritto 140 in vector[12] all'indirizzo 0x7ffcd660370
[14]: 150
> Scritto 150 in vector[13] all'indirizzo 0x7ffcd660374
[15]: 1
> Scritto 1 in vector[14] all'indirizzo 0x7ffcd660378
[16]: 6
> Scritto 6 in vector[15] all'indirizzo 0x7ffcd66037c
[17]: 10
> Scritto 10 in vector[16] all'indirizzo 0x7ffcd660380
[18]: 89
> Scritto 89 in vector[17] all'indirizzo 0x7ffcd660384
[19]: 5542
> Scritto 5542 in vector[18] all'indirizzo 0x7ffcd660388
[20]: aaa
> Scritto 5542 in vector[19] all'indirizzo 0x7ffcd66038c
[21]: 5542
> Scritto 5542 in vector[20] all'indirizzo 0x7ffcd660390
[22]: 5542
> Scritto 5542 in vector[21] all'indirizzo 0x7ffcd660394
```

```
Session Actions Edit View Help
kali@kali: ~$ ./bof.o
[1794]: >> Scritto 5542 in vector[1793] all'indirizzo 0x7ffcd661f64
[1795]: >> Scritto 5542 in vector[1794] all'indirizzo 0x7ffcd661f68
[1796]: >> Scritto 5542 in vector[1795] all'indirizzo 0x7ffcd661f6c
[1797]: >> Scritto 5542 in vector[1796] all'indirizzo 0x7ffcd661f70
[1798]: >> Scritto 5542 in vector[1797] all'indirizzo 0x7ffcd661f74
[1799]: >> Scritto 5542 in vector[1798] all'indirizzo 0x7ffcd661f78
[1800]: >> Scritto 5542 in vector[1799] all'indirizzo 0x7ffcd661f7c
[1801]: >> Scritto 5542 in vector[1800] all'indirizzo 0x7ffcd661f80
[1802]: >> Scritto 5542 in vector[1801] all'indirizzo 0x7ffcd661f84
[1803]: >> Scritto 5542 in vector[1802] all'indirizzo 0x7ffcd661f88
[1804]: >> Scritto 5542 in vector[1803] all'indirizzo 0x7ffcd661f8c
[1805]: >> Scritto 5542 in vector[1804] all'indirizzo 0x7ffcd661f90
[1806]: >> Scritto 5542 in vector[1805] all'indirizzo 0x7ffcd661f94
[1807]: >> Scritto 5542 in vector[1806] all'indirizzo 0x7ffcd661f98
[1808]: >> Scritto 5542 in vector[1807] all'indirizzo 0x7ffcd661fa0
[1809]: >> Scritto 5542 in vector[1808] all'indirizzo 0x7ffcd661fa4
[1810]: >> Scritto 5542 in vector[1809] all'indirizzo 0x7ffcd661fa8
[1811]: >> Scritto 5542 in vector[1810] all'indirizzo 0x7ffcd661fab
[1812]: >> Scritto 5542 in vector[1811] all'indirizzo 0x7ffcd661fac
[1813]: >> Scritto 5542 in vector[1812] all'indirizzo 0x7ffcd661fad
[1814]: >> Scritto 5542 in vector[1813] all'indirizzo 0x7ffcd661fae
[1815]: >> Scritto 5542 in vector[1814] all'indirizzo 0x7ffcd661faf
[1816]: >> Scritto 5542 in vector[1815] all'indirizzo 0x7ffcd661fb0
[1817]: >> Scritto 5542 in vector[1816] all'indirizzo 0x7ffcd661fb4
[1818]: >> Scritto 5542 in vector[1817] all'indirizzo 0x7ffcd661fb8
[1819]: >> Scritto 5542 in vector[1818] all'indirizzo 0x7ffcd661fbc
[1820]: >> Scritto 5542 in vector[1819] all'indirizzo 0x7ffcd661fb0
[1821]: >> Scritto 5542 in vector[1820] all'indirizzo 0x7ffcd661fb4
[1822]: >> Scritto 5542 in vector[1821] all'indirizzo 0x7ffcd661fb8
[1823]: >> Scritto 5542 in vector[1822] all'indirizzo 0x7ffcd661fbc
[1824]: >> Scritto 5542 in vector[1823] all'indirizzo 0x7ffcd661fb0
[1825]: >> Scritto 5542 in vector[1824] all'indirizzo 0x7ffcd661fb4
[1826]: >> Scritto 5542 in vector[1825] all'indirizzo 0x7ffcd661fb8
[1827]: >> Scritto 5542 in vector[1826] all'indirizzo 0x7ffcd661fbc
[1828]: >> Scritto 5542 in vector[1827] all'indirizzo 0x7ffcd661fb0
[1829]: >> Scritto 5542 in vector[1828] all'indirizzo 0x7ffcd661fb4
[1830]: >> Scritto 5542 in vector[1829] all'indirizzo 0x7ffcd661fb8
[1831]: >> Scritto 5542 in vector[1830] all'indirizzo 0x7ffcd661fbc
[1832]: >> Scritto 5542 in vector[1831] all'indirizzo 0x7ffcd661fb0
[1833]: >> Scritto 5542 in vector[1832] all'indirizzo 0x7ffcd661fb4
[1834]: >> Scritto 5542 in vector[1833] all'indirizzo 0x7ffcd661fb8
[1835]: >> Scritto 5542 in vector[1834] all'indirizzo 0x7ffcd661fbc
[1836]: >> Scritto 5542 in vector[1835] all'indirizzo 0x7ffcd661fb0
[1837]: >> Scritto 5542 in vector[1836] all'indirizzo 0x7ffcd661fb4
[1838]: >> Scritto 5542 in vector[1837] all'indirizzo 0x7ffcd661fb8
[1839]: >> Scritto 5542 in vector[1838] all'indirizzo 0x7ffcd661fbc
[1840]: >> Scritto 5542 in vector[1839] all'indirizzo 0x7ffcd661fb0
zsh: segmentation fault ./bof.o
```

Esecuzione del programma `bof.o` compilato dal file `bof.c`. Si può notare la prima manifestazione di *buffer overflow* alla 10ma richiesta di input. In più si può notare che alla ricezione dell'input nella 12ma richiesta, si è sovrascritto l'inirizzo di memoria della variabile `i` (`0x7ffcdf660548`) spostando l'indice a `132`

Continuazione dell'esecuzione del programma e termine per errore di segmentazione (*Segmentation Fault*), causato dall'inserimento di un input non numerico (è stata inserita la sequenza di caratteri `aaa`)

Segue lo stack di esecuzione del programma, e della manifestazione del BOF:



4. Mitigazione del BOF

Per dimostrazione delle modifiche di mitigazione del BOF, è stato creato un nuovo file "`no_bof.c`", che presenta controlli e convalide dell'utente. Sono implementati i seguenti controlli:

Controllo	Descrizione	Cosa previene
Limite sul ciclo	<code>for (i = 0; i < DIM; i++)</code>	Impedisce di scrivere oltre l'array
Validazione del tipo di dato	<code>scanf</code> restituisce <code>1</code> se ha letto un intero	Impedisce l'inserimento di lettere o caratteri speciali
Pulizia del buffer	<code>while(getchar() != '\n')</code>	Rimuove caratteri residui dopo input non valido
Conteggio effettivo	<code>input_count</code>	Doppia verifica che il numero di input sia corretto

Segeue il codice complessivo per la mitigazione del BOF:

```
1 #include <stdio.h>
2 #include <stdlib.h> // per exit()
3
4 #define DIM 10
5
6 int main() {
7     int vector[DIM], i, j, k;
8     int swap_var;
9     int input_count = 0;
10    int result;
11
12    printf("Versione SICURA - con controlli di input\n\n");
13    printf("Inserire %d interi:\n", DIM);
14
15    /* CONTROLLO 1: limite rigido sul numero di input */
16    for (i = 0; i < DIM; i++) {
17        int c = i + 1;
18        printf("[%d]: ", c);
19
20        /* CONTROLLO 2: validazione del tipo di dato */
21        result = scanf("%d", &vector[i]);
22
23        if (result != 1) {
24            /* Input non numerico: pulisci il buffer e richiedi */
25            printf("ERRORE: Inserire un numero intero valido.\n");
26            /* Svuota il buffer di input fino al newline */
27            while (getchar() != '\n');
28            i--; // Ripeti la stessa posizione
29            continue;
30        }
31
32        input_count++;
33    }
34
35    /* CONTROLLO 3: verifica che siano stati inseriti esattamente DIM valori */
36    if (input_count != DIM) {
37        fprintf(stderr, "ERRORE GRAVE: Il numero di elementi inseriti (%d) "
38            "non corrisponde a quanto previsto (%d).\n",
39            input_count, DIM);
40        return 1;
41    }
42
43    /* Stampa vettore */
44    printf("\nIl vettore inserito e':\n");
45    for (i = 0; i < DIM; i++) {
46        printf("[%d]: %d\n", i + 1, vector[i]);
47    }
48
49    /* Bubble Sort */
50    for (j = 0; j < DIM - 1; j++) {
51        for (k = 0; k < DIM - j - 1; k++) {
52            if (vector[k] > vector[k + 1]) {
53                swap_var = vector[k];
54                vector[k] = vector[k + 1];
55                vector[k + 1] = swap_var;
56            }
57        }
58    }
59}
```

```
--  
60  
61     /* Stampa ordinata */  
62     printf("\nIl vettore ordinato e':\n");  
63     for (j = 0; j < DIM; j++) {  
64         printf("[%d]: %d\n", j + 1, vector[j]);  
65     }  
66     return 0;  
}
```

5. Conclusioni

Con questo laboratorio si è dimostrato come una mancata validazione degli indici di accesso a un array permetta la scrittura oltre la memoria allocata, portando a un errore di *Segmentation Fault* per corruzione dello stack frame.